SIMATS ENGINEERING

CO1-ASSESSMENT TOOL 1-Constrain Based Problem

COURSE NAME: R PROGRAMMING **COURSE CODE: ITA04**

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.
(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

QUESTIONS: **Total Marks: 100**

1. **Student Mark Analysis using Vectors**
   Create an R program to store the marks of 10 students in a vector. Perform the following operations:
   a) Display the marks
   b) Calculate total and average
   c) Find maximum and minimum marks
   d) Display the marks greater than 50 using logical-vector operations
   e) Display the position of the highest mark.

2. **Student Information using Different Data Types**
   Develop an R program to store student information such as student name, register number, age, gender, marks, and pass/fail status using appropriate R data types. Display each variable, its value, and its data type. Explain the suitability of each data type.

3. **Matrix Operations in R**
   Create two 3×3matrices representing marks obtained by students in two different subjects. Write an R program to perform matrix addition, subtraction, multiplication, transpose, and extraction of selected elements. Explain the operations with suitable output.

4. **Employee Salary Analysis using Lists**
   Create an R list containing an employee's name, employee ID, department, basic salary, allowances, and deductions. Write an R program to calculate the gross salary and net salary and display the individual components of the list.

5. **Product Sales using Arrays**
   Create a three-dimensional array to store the sales of three products for two branches over two months. Write an R program to create the array, assign suitable dimension names, access individual values, and calculate the total sales.

6. **Student Grade Analysis using Factors**
   Create a factor containing the grades obtained by 10 students (`A`, `B`, `C`, `D`). Write an R program to display the factor levels, count the number of students in each grade, and access individual factor elements. Explain why a factor is suitable for representing categorical data.

7. **Student Data Frame Problem**
   Create a data frame containing **Student ID, Name, Age, Department, Gender, and Marks** for five students. Write an R program to create the data frame, display its contents, access individual columns and rows, and calculate the average, maximum, and minimum marks.

8. **Logical Vector-Based Student Selection**
   Create a vector containing the marks of 15 students. Using logical-vector operations, write an R program to identify students who:
   a) scored above 75
   b) scored between 50 and 75
   c) scored below 50
   d) passed the examination.
   Display the corresponding marks for each category.

9. **R Data Structure Problem**
   Develop an R program to represent student information using **vectors, lists, matrices, factors, and data frames**. Demonstrate how each structure is created and accessed. Explain which type of information is most suitable for each data structure.
10. • **R Calculation and Data Structure Problem**
    Write an R program for a small student result-processing application. Store the marks of five students in vectors, calculate total and average marks, create a result matrix, assign grades using factors, and organize the complete student information in a data frame. Display the final result.

Code of Conduct:

I Hasya Sai Reddy Venati (192420010) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	**20%**	**Identifies all inputs, outputs, constraints accurately**	**Minor missing elements**	**Partial understanding**	**Incorrect interpretation**
Algorithm Design	**30%**	**Complete, correct, and logically sound solution**	**Minor logical gaps**	**Partially correct logic**	**Incorrect approach**
Use of Control Structures	**20%**	**Efficient and appropriate use of loops, conditions, functions/modules**	**Minor inefficiencies**	**Limited or basic usage**	**Incorrect or missing constructs**
Pseudocode Structure	**10%**	**Well-structured, clear, consistent format, easy to follow**	**Mostly clear with minor issues**	**Difficult to follow in parts**	**Poorly structured and unclear**
Completeness	**20%**	**Handles all cases; optimized approach**	**Handles most cases; reasonable efficiency**	**Handles basic cases only**	**Incomplete or inefficient**

Q1.
Input:

```
marks <- c(45, 67, 82, 56, 91, 38, 74, 63, 88, 52)

print(marks)
print(sum(marks))
print(mean(marks))
print(max(marks))
print(min(marks))
print(marks[marks > 50])
print(which.max(marks))
```

Output:

```
> marks=c(45, 67, 82, 56, 91, 38, 74, 63, 88, 52)
> print(which.max(marks))
[1] 5
```

Q2.
Input:

```
student_name <- "Arun"
register_number <- 101
age <- 20
gender <- "Male"
marks <- 78.5
status <- TRUE

print(student_name)
print(register_number)
print(age)
print(gender)
print(marks)
print(status)

print(class(student_name))
print(class(register_number))
print(class(age))
print(class(gender))
print(class(marks))
print(class(status))
```

Output:

```
> print(student_name)
[1] "Arun"
> print(register_number)
[1] 101
> print(age)
[1] 20
> print(gender)
[1] "Male"
> print(marks)
[1] 78.5
> print(status)
[1] TRUE
> print(class(student_name))
[1] "character"
> print(class(register_number))
[1] "numeric"
> print(class(age))
[1] "numeric"
> print(class(gender))
[1] "character"
> print(class(marks))
[1] "numeric"
> print(class(status))
[1] "logical"
> source("~/.active-rstudio-document")
[1] "Arun"
```

Q3.
Input:

```
matrix1 <- matrix(c(10, 20, 30, 40), nrow = 2, ncol = 2)
matrix2 <- matrix(c(5, 10, 15, 20), nrow = 2, ncol = 2)

print(matrix1)
print(matrix2)

print(matrix1 + matrix2)
print(matrix1 - matrix2)
print(matrix1 %*% matrix2)
print(t(matrix1))

print(matrix1[1, 2])
print(matrix1[2, ])
print(matrix1[, 1])
```

Output:

```
> print(matrix1)
     [,1] [,2]
[1,]   10   30
[2,]   20   40
> print(matrix2)
     [,1] [,2]
[1,]    5   15
[2,]   10   20
>
> print(matrix1 + matrix2)
     [,1] [,2]
[1,]   15   45
[2,]   30   60
```

```
> print(matrix1 - matrix2)
     [,1] [,2]
[1,]    5   15
[2,]   10   20
> print(matrix1 %*% matrix2)
     [,1] [,2]
[1,]  350  750
[2,]  500 1100
> print(t(matrix1))
     [,1] [,2]
[1,]   10   20
[2,]   30   40
>
```

```
> print(matrix1[1, 2])
[1] 30
> print(matrix1[2, ])
[1] 20 40
> print(matrix1[, 1])
[1] 10 20
> print(matrix1[, 1])
```

Q4.
Input:

```
emp <- list(name="Arun",id=101,dept="IT",
        basic=30000,allowance=5000,deduction=3000)

gross <- emp$basic+emp$allowance
net <- gross-emp$deduction

print(emp)
print(gross)
print(net)
```

Output:

```
$name
[1] "Arun"

$id
[1] 101

$dept
[1] "IT"

$basic
[1] 30000

$allowance
[1] 5000
```

```
$basic
[1] 30000

$allowance
[1] 5000

$deduction
[1] 3000

> print(gross)
[1] 35000
> print(net)
[1] 32000
> print(matrix1[, 1])
```

Q5.
Input:

```
sales <- array(c(100,120,150,130,200,180,170,160,90,110,140,125),
               dim=c(3,2,2))

print(sales)
print(sales[1,1,1])
print(sum(sales))
```

Output:

```
> print(sales)
, , 1

     [,1] [,2]
[1,]  100  130
[2,]  120  200
[3,]  150  180

, , 2

     [,1] [,2]
[1,]  170  110
[2,]  160  140
[3,]   90  125
```

```
> print(sales[1,1,1])
[1] 100
> print(sum(sales))
[1] 1675
> print(matrix1[, 1])
```

Q6.
Input:

```
grades <- factor(c("A","B","A","C","D","B","A","C","B","D"))

print(grades)
```

print(levels(grades))
print(table(grades))
print(grades[1])

Output:

```
> print(grades)
 [1] A B A C D B A C B D
Levels: A B C D
> print(levels(grades))
[1] "A" "B" "C" "D"
> print(table(grades))
grades
A B C D
3 3 2 2
> print(grades[1])
[1] A
Levels: A B C D
> print(matrix1[, 1])
```

Q7.
Input:

```
students <- data.frame(
ID=c(101,102,103,104,105),
Name=c("Arun","Bala","Charan","Deepak","Elan"),
Age=c(20,21,19,20,22),
Dept=c("IT","CSE","ECE","IT","CSE"),
Gender=c("M","M","M","M","M"),
Marks=c(78,85,67,92,74))

print(students)
print(students$Name)
print(students[1,])
print(mean(students$Marks))
print(max(students$Marks))
print(min(students$Marks))
```

Output:

```
> print(students)
   ID   Name Age Dept Gender Marks
1 101   Arun  20   IT      M    78
2 102   Bala  21  CSE      M    85
3 103 Charan  19  ECE      M    67
4 104 Deepak  20   IT      M    92
5 105   Elan  22  CSE      M    74
> print(students$Name)
[1] "Arun"   "Bala"   "Charan" "Deepak" "Elan"
> print(students[1,])
   ID Name Age Dept Gender Marks
1 101 Arun  20   IT      M    78
> print(mean(students$Marks))
[1] 79.2
```

```
> print(mean(students$Marks))
[1] 79.2
> print(max(students$Marks))
[1] 92
> print(min(students$Marks))
[1] 67
> print(matrix1[, 1])
```

Q8.
Input:

```
marks <- c(45,82,67,39,91,55,73,48,88,62,35,76,50,69,95)

print(marks[marks>75])
print(marks[marks>=50 & marks<=75])
print(marks[marks<50])
print(marks[marks>=50])
```

Output:

```
> print(marks[marks>75])
[1] 82 91 88 76 95
> print(marks[marks>=50 & marks<=75])
[1] 67 55 73 62 50 69
> print(marks[marks<50])
[1] 45 39 48 35
> print(marks[marks>=50])
 [1] 82 67 91 55 73 88 62 76 50 69 95
> print(matrix1[, 1])
```

Q9.
Input:

```
v <- c("Arun","Bala","Charan")
l <- list(Name="Arun",Age=20,Marks=85)
m <- matrix(1:6,2,3)
f <- factor(c("A","B","A","C"))
d <- data.frame(ID=1:3,Name=v,Marks=c(80,70,90))

print(v)
print(l)
print(m)
print(f)
print(d)
```

Output:

```
> print(v)
[1] "Arun"   "Bala"   "Charan"
> print(l)
$Name
[1] "Arun"

$Age
[1] 20

$Marks
[1] 85
```

```
> print(m)
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> print(f)
[1] A B A C
Levels: A B C
> print(d)
  ID   Name Marks
1  1   Arun    80
2  2   Bala    70
3  3 Charan    90
> print(matrix1[, 1])
```

Q10.
Input:

```
name <- c("Arun","Bala","Charan","Deepak","Elan")
m <- c(80,75,90,65,85)
s <- c(85,70,88,72,80)
e <- c(78,82,84,68,86)

total <- m+s+e
avg <- total/3

grade <- factor(ifelse(avg>=85,"A",
            ifelse(avg>=70,"B",
            ifelse(avg>=50,"C","D"))))

result <- data.frame(name,m,s,e,total,avg,grade)

print(result)
```

Output:

```
> print(result)
    name  m  s  e total      avg grade
1   Arun 80 85 78   243 81.00000     B
2   Bala 75 70 82   227 75.66667     B
3 Charan 90 88 84   262 87.33333     A
4 Deepak 65 72 68   205 68.33333     C
5   Elan 85 80 86   251 83.66667     B
```